

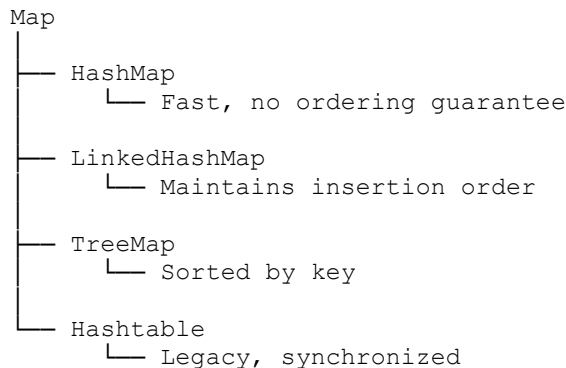
Yes. Since you **already understand maps conceptually** and your actual problem is *syntax/property recall under interview/contest pressure*, I would not teach this as a beginner lesson.

We'll treat it as a **Java Collections recall drill**.

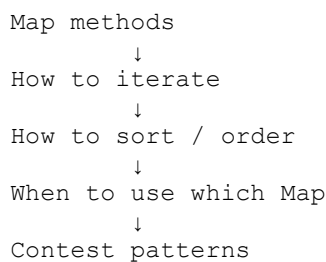
Your DSA repository already has a dedicated Hashing section, including `InternalHashMap.java`, `CountDistinct.java`, `MajorityElt.java`, `SubarraySumK.java`, etc., so maps are clearly part of your existing practice rather than a new topic. You also specifically prefer Java for DSA.

The approach

I want you to build one mental hierarchy:



And separately:



The important part is **not memorizing 30 methods**. You need to be extremely fluent with ~10–15 things.

1. The three Maps you actually care about

For DSA, focus heavily on these:

Map	Ordering	Typical complexity	Main use
HashMap	None	$O(1)$ average	Frequency/counting, lookup
LinkedHashMap	Insertion order	$O(1)$ average	Map + preserve insertion order
TreeMap	Sorted by key	$O(\log n)$	Ordered keys, min/max key, ranges

`Hashtable` exists, but **don't spend revision time on it for DSA**. It's mostly legacy Java knowledge.

2. HashMap — your default Map

The syntax you should be able to write without thinking:

```
Map<Integer, Integer> map = new HashMap<>();
```

I recommend declaring against the interface:

```
Map<Integer, Integer> map = new HashMap<>();
```

rather than:

```
HashMap<Integer, Integer> map = new HashMap<>();
```

unless you specifically need `HashMap`-specific behavior.

Basic operations

```
map.put(key, value);
map.get(key);
map.containsKey(key);
map.containsValue(value);
map.remove(key);
map.size();
map.isEmpty();
map.clear();
```

The ones you **really need burned into muscle memory**:

```
put()
get()
getOrDefault()
containsKey()
remove()
size()
```

3. The most important frequency-map pattern

This:

```
if (map.containsKey(x)) {
    map.put(x, map.get(x) + 1);
} else {
    map.put(x, 1);
}
```

works, but you should almost always write:

```
map.put(x, map.getOrDefault(x, 0) + 1);
```

This is one of those tiny syntax improvements that matters enormously in contests.

Example:

```
int[] nums = {1, 2, 2, 3, 1, 2};
```

```
Map<Integer, Integer> freq = new HashMap<>();

for (int x : nums) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
}
```

Result conceptually:

```
1 → 2
2 → 3
3 → 1
```

4. `get()` VS `getOrDefault()`

This is a common source of mistakes.

```
map.get(x)
```

returns:

```
value      → if key exists
null       → if key doesn't exist
```

Whereas:

```
map.getOrDefault(x, 0)
```

returns:

```
value      → if key exists
0          → if key doesn't exist
```

So:

```
map.get(x) + 1
```

can fail if `x` isn't present.

But:

```
map.getOrDefault(x, 0) + 1
```

is safe.

5. Iteration — this is where I want you to become fluent

There are **three important ways**.

A. Keys only

```
for (int key : map.keySet()) {  
    System.out.println(key);  
}
```

Think:

`keySet()` → give me keys.

B. Values only

```
for (int value : map.values()) {  
    System.out.println(value);  
}
```

Think:

`values()` → give me values.

C. Key + value — most important

```
for (Map.Entry<Integer, Integer> entry : map.entrySet()) {  
    int key = entry.getKey();  
    int value = entry.getValue();  
  
    System.out.println(key + " " + value);  
}
```

Think:

`entrySet()` → give me key-value pairs.

This should become automatic:

```
for (Map.Entry<K, V> entry : map.entrySet()) {  
    K key = entry.getKey();  
    V value = entry.getValue();  
}
```

You don't need to memorize some alternative fancy syntax.

6. Java also allows this

You will frequently see:

```
for (var entry : map.entrySet()) {  
    System.out.println(entry.getKey() + " " + entry.getValue());  
}
```

Or:

```
map.forEach((key, value) -> {  
    System.out.println(key + " " + value);  
});
```

These are useful, but for **DSA interview coding**, I would prioritize:

```
for (Map.Entry<Integer, Integer> entry : map.entrySet())
```

because it is explicit and universally familiar.

7. `containsKey()` VS `containsValue()`

This distinction should be instant:

```
map.containsKey(x)
```

→ Does this **key** exist?

```
map.containsValue(x)
```

→ Does this **value** exist?

In DSA, `containsKey()` is vastly more common.

8. `remove()`

Simple:

```
map.remove(key);
```

It removes the mapping for that key.

You can also do:

```
map.remove(key, value);
```

which removes it **only if the key currently maps to that value**.

Less common in DSA, but worth knowing.

9. Updating values

Suppose:

$A \rightarrow 5$

Then:

```
map.put("A", 10);
```

changes it to:

$A \rightarrow 10$

`put()` is both:

insert **and** update.

That's worth remembering.

10. `putIfAbsent()`

This one is easy to forget:

```
map.putIfAbsent(key, value);
```

Meaning:

Insert only if the key doesn't already exist.

Example:

```
map.put("A", 10);  
map.putIfAbsent("A", 20);
```

Result:

$A \rightarrow 10$

Not:

$A \rightarrow 20$

This is useful, but much less important than `put()` and `getOrDefault()` for DSA.

11. TreeMap

Now the important difference.

```
Map<Integer, String> map = new TreeMap<>();
```

If you insert:

```
map.put(50, "A");  
map.put(10, "B");  
map.put(30, "C");  
map.put(20, "D");
```

iteration gives keys in sorted order:

10
20
30
50

So:

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {  
    System.out.println(entry.getKey() + " " + entry.getValue());  
}
```

is sorted by key.

This is the **primary reason you reach for TreeMap**.

12. TreeMap methods you should remember

This is where TreeMap becomes significantly more useful than HashMap.

Suppose:

```
TreeMap<Integer, String> map = new TreeMap<>();
```

Then:

```
map.firstKey();  
map.lastKey();
```

Give:

```
smallest key  
largest key
```

And:

```
map.firstEntry();  
map.lastEntry();
```

give the corresponding key-value pairs.

The extremely useful navigational methods

```
map.lowerKey(x);  
map.floorKey(x);  
map.ceilingKey(x);  
map.higherKey(x);
```

Memorize the four-word sequence:

```
lower  
floor  
ceiling  
higher
```

For key x :

```
lower(x)    < x
floor(x)    <= x
ceiling(x)  >= x
higher(x)   > x
```

This is a **very useful contest pattern**.

Example keys:

```
10  20  30  40
```

For:

```
x = 25
```

you get:

```
lowerKey(25)  → 20
floorKey(25)  → 20
ceilingKey(25) → 30
higherKey(25) → 30
```

For $x = 30$:

```
lowerKey(30)  → 20
floorKey(30)  → 30
ceilingKey(30) → 30
higherKey(30) → 40
```

That's the cleanest way to remember the difference.

13. LinkedHashMap

Syntax:

```
Map<Integer, Integer> map = new LinkedHashMap<>();
```

The key property:

Preserves insertion order.

If:

```
map.put(5, 100);
map.put(2, 200);
map.put(8, 300);
```

iteration remains:

```
5
2
8
```

It is basically useful when you want:

HashMap-like lookup
+
predictable insertion-order iteration

For ordinary DSA problems, you'll encounter it much less than `HashMap`.

14. A Map cheat sheet worth actually memorizing

This is the compact version I'd want you to eventually recall:

```
Map<K, V> map = new HashMap<>();
```

Core

```
map.put(k, v);  
map.get(k);  
map.getDefault(k, defaultValue);  
map.containsKey(k);  
map.containsValue(v);  
map.remove(k);  
map.size();  
map.isEmpty();
```

Iteration

```
for (K k : map.keySet()) { }  
  
for (V v : map.values()) { }  
  
for (Map.Entry<K, V> e : map.entrySet()) {  
    e.getKey();  
    e.getValue();  
}
```

Useful updates

```
map.putIfAbsent(k, v);  
map.replace(k, v);
```

TreeMap

```
TreeMap<K, V> map = new TreeMap<>();  
  
map.firstKey();  
map.lastKey();  
  
map.lowerKey(k);  
map.floorKey(k);  
map.ceilingKey(k);  
map.higherKey(k);
```

15. One subtle thing you absolutely should know

`HashMap` does NOT guarantee ordering.

Never write logic that assumes:

HashMap

will iterate in:

- insertion order
- sorted order
- any predictable order

Even if it *appears* to do so during your testing.

If you need:

insertion order → LinkedHashMap
sorted key order → TreeMap
no ordering requirement → HashMap

That's the decision tree.

16. Your actual revision should now be active recall

Rather than me dumping every Map method, let's do this like a **contest/interview recall session**.

I would structure the rest as:

Round 1 — Syntax recall

I'll give you situations and you write the Java statement.

Example:

Create a map from `Integer` to `Integer`.

You answer:

```
Map<Integer, Integer> map = new HashMap<>();
```

Then:

Increment the frequency of `x`, assuming absent means 0.

You answer:

```
map.put(x, map.getOrDefault(x, 0) + 1);
```

Round 2 — Iteration recall

I'll deliberately mix:

- keys
- values
- key + value

- `TreeMap`

and make you choose the correct syntax.

Round 3 — Map selection

I'll give you problems like:

Need frequency counting.

→ `HashMap`

Need keys automatically sorted.

→ `TreeMap`

Need insertion order preserved.

→ `LinkedHashMap`

Round 4 — `TreeMap` navigation

This is where we'll drill:

```
lower  
floor  
ceiling  
higher
```

because these are exactly the methods that disappear from memory when you haven't used them recently.

Round 5 — DSA patterns

Finally we'll connect them to things you've already done in your `Hashing` section—frequency counting, distinct elements, zero-sum subarrays, subarray sum, etc.—rather than treating Collections as isolated Java syntax.

Let's start with Round 1. No notes.

Question 1

You have:

```
int[] nums = {1, 2, 2, 3, 1, 2};
```

Write Java code to create a frequency map.

Only write the code.